

Backlog dev priorise code source

Priorite 1

Corriger l'email du formulaire pre-session landing-solo

- Impact : eleve
- Pourquoi : un prospect peut remplir le bon champ visible, mais la validation, le bouton et la sauvegarde ne lisent pas toujours la meme cle
- Fichiers a regarder :

`src/components/FormulairePréSession.tsx`

`src/services/preSessionFormService.ts`

- Ce qu'il faut verifier :

le champ visible, la validation, l'objet envoye et la reprise des donnees

- Methode de resolution pas a pas :
 1. choisir `clientEmail` comme cle unique
 2. remplacer les lectures `responses.email` liees a ce parcours
 3. aligner validation, bouton et persistance
 4. verifier les cas deja saisis si une ancienne cle `email` existe
 5. ajouter un test simple sur le parcours

- Definition de fini :

une seule cle email est utilisee du debut a la fin

Rendre le formulaire contact fiable sans Crisp

- Impact : eleve
- Pourquoi : un message client ne doit pas dependre d'un script tiers charge cote navigateur
- Fichiers a regarder :

`src/pages/ContactPage.tsx`

`src/services/crispService.ts`

- Ce qu'il faut verifier :

comportement sans Crisp, stockage du message, message de succes reel

- Methode de resolution pas a pas :
 1. definir un canal principal fiable
 2. garder Crisp comme option complementaire
 3. n'afficher le succes qu'apres une vraie confirmation d'envoi
 4. tracer l'erreur et proposer une issue claire a l'utilisateur
 5. couvrir le cas nominal et le cas Crisp indisponible
- Definition de fini :

le message part dans un canal fiable meme si Crisp ne charge pas

Fiabiliser la confirmation apres commande

- Impact : eleve
- Pourquoi : la page ne doit pas afficher un succes complet sur la seule base de l'URL ou du navigateur
- Fichiers a regarder :

`src/pages/ConfirmationPage.tsx`

`src/pages/OrderConfirmationPage.tsx`

- Ce qu'il faut verifier :

id de commande de reference, relecture cote serveur, etat de chargement, etat d'erreur

- Methode de resolution pas a pas :
 1. ne garder dans l'URL qu'un identifiant de reference
 2. relire la commande cote serveur ou base de donnees
 3. verifier les statuts avant affichage
 4. afficher un ecran d'attente pendant controle
 5. gerer proprement les cas introuvables ou incohérents
- Definition de fini :

la confirmation s'appuie sur une commande relue et verifiee

Sortir la validation finale du paiement du navigateur

- Impact : critique
- Pourquoi : le front ne doit pas pouvoir decider seul que la commande est `paid`
- Fichiers a regarder :

`src/components/OrderValidation.tsx`

`src/config/supabase.ts`

- Ce qu'il faut verifier :

qui met a jour `payment_status` , sur quelle preuve, via quel point serveur

- Methode de resolution pas a pas :
 1. creer une verification serveur ou webhook
 2. faire verifier la preuve de paiement hors navigateur
 3. mettre a jour `orders` seulement apres verification
 4. faire du front un simple ecran d'attente et de retour
 5. tracer les changements de statut
- Definition de fini :

le statut de paiement est mis a jour seulement apres verification serveur

Revoir les policies SQL trop ouvertes

- Impact : critique
- Pourquoi : plusieurs scripts du depot autorisent des lectures ou operations trop larges
- Fichiers a regarder :

`sql/create_pre_session_forms.sql`

`sql/pre_session_forms_migration.sql`

`sql/create_live_sessions.sql`

- Ce qu'il faut verifier :

lecture publique, acces trop larges, compte authentifie trop permissif

- Methode de resolution pas a pas :
 1. lister les besoins reels par table
 2. supprimer les policies globales ouvertes
 3. separer acces public, acces authentifie et acces admin
 4. verifier les droits storage associes

5. tester les droits reellement deployes

- Definition de fini :

orders, pre-session, live sessions et images n'ont plus de policy trop ouverte

Priorite 2

Renforcer l'accès admin

- Impact : eleve
- Pourquoi : la simple presence d'une session ne suffit pas a montrer un vrai controle de role
- Fichiers a regarder :

```
src/pages/AdminLogin.tsx
```

```
src/pages/AdminDashboard.tsx
```

politiques Supabase reelles

- Methode de resolution pas a pas :
 1. definir le role admin attendu
 2. verifier ce role au login et sur les donnees
 3. refuser l'accès si la session existe sans bon role
 4. tracer les refus utiles
- Definition de fini :

l'accès admin depend d'un vrai role ou claim controle

Assainir configuration, valeurs en dur et identifiants tiers

- Impact : eleve
- Pourquoi : plusieurs valeurs importantes restent en dur ou dupliques, ce qui complique la maintenance et augmente le risque d'erreur
- Fichiers a regarder :

```
src/App.tsx
```

```
src/config/supabase.ts
```

```
src/components/PayPalButton.tsx
```

```
.env
```

- Ce qu'il faut verifier :

ID Crisp, emails metier, modes de paiement, valeurs de fallback, variables publiques vs valeurs en dur

- Methode de resolution pas a pas :
 1. lister les valeurs metier et techniques encore en dur
 2. separer ce qui doit vivre en config, en env public ou en constante de design
 3. retirer les doublons entre code et configuration
 4. documenter ce qui est public et ce qui ne doit jamais etre cote front
 5. tester chaque environnement apres nettoyage
- Definition de fini :

les identifiants tiers, valeurs metier globales et modes de configuration ont une source unique claire

Reduire les logs sensibles et l'exposition navigateur

- Impact : eleve
- Pourquoi : plusieurs logs affichent trop d'informations sur la configuration, les paiements ou les payloads
- Fichiers a regarder :

```
src/config/supabase.ts
```

```
src/components/PayPalButton.tsx
```

```
src/pages/ContactPage.tsx
```

- Ce qu'il faut verifier :

logs de config, logs de paiement, payloads clients, details commandes

- Methode de resolution pas a pas :
 1. lister les logs de debug encore presents
 2. retirer ceux qui exposent donnees ou configuration
 3. garder seulement des logs utiles, courts et non sensibles
 4. reserver le debug detaille a un mode local maitrise
- Definition de fini :

aucun log sensible inutile n'est expose dans le navigateur

Reduire la dette type/lint sur les parcours critiques

- Impact : eleve
- Pourquoi : le lint en echec ralentit toutes les corrections suivantes
- Fichiers a regarder :

src/pages/ConfiguratorPage.tsx

src/components/ConfigurationCaptureModalAdaptive.tsx

src/components/FormulairePréSession.tsx

src/components/OrderValidation.tsx

src/types/orders.ts

src/types/preSession.ts

- Methode de resolution pas a pas :
 1. traiter d'abord les types partages
 2. remplacer les `any` structurants
 3. corriger les warnings `useEffect`
 4. viser d'abord les flux business critiques
 5. descendre par paliers mesurables
- Definition de fini :

baisse nette des erreurs lint sur les ecrans business

Poser un premier filet de securite de tests

- Impact : moyen a eleve
- Pourquoi : aucun socle de test clair n'est visible aujourd'hui
- A couvrir d'abord :

pre-session

contact

confirmation

validation commande

- Methode de resolution pas a pas :
 1. installer un socle de test simple
 2. ecrire un test nominal par parcours
 3. ecrire un test d'erreur par parcours

4. brancher ces tests au travail quotidien

- Definition de fini :

au moins un test nominal et un test d'erreur sur chaque parcours critique

Revoir l'exposition des donnees dans le navigateur

- Impact : eleve
- Pourquoi : `localStorage` , IDs URL et `publicUrl` exposent plus que necessaire
- Fichiers a regarder :

`src/pages/ConfirmationPage.tsx`

`src/pages/ConfiguratorPage.tsx`

`src/config/supabase.ts`

- Methode de resolution pas a pas :
 1. lister les donnees stockees cote navigateur
 2. garder seulement le strict utile
 3. reduire les identifiants visibles dans les URL
 4. revoir le besoin reel de `publicUrl`
- Definition de fini :

moins de donnees sensibles persistent dans le navigateur

Priorite 3

Reprendre l'incoherence de socle front

- Impact : moyen a eleve
- Pourquoi : le projet a deja des tokens, mais ils sont contournes par des styles inline, des fallback en dur et des overrides locaux
- Fichiers a regarder :

`src/styles/tokens.css`

`src/pages/Home.tsx`

`src/pages/ContactPage.tsx`

`src/pages/ConfiguratorPage.tsx`

`src/components/ConfigurationCaptureModalAdaptive.tsx`

- Ce qu'il faut verifier :

source de verite des couleurs, polices, espacements, radius, ombres et transitions

- Methode de resolution pas a pas :
 1. figer les tokens officiels
 2. retirer les doubles definitions et les overrides inutiles
 3. remplacer les styles inline repetes par des classes ou composants
 4. verifier les pages principales apres harmonisation
- Definition de fini :

le socle front a une source de verite unique et lisible

Reprendre la dette de coherence UI / design system

- Impact : moyen
- Pourquoi : plusieurs nuances, polices, boutons et comportements visuels coexistent sans regle assez claire
- Fichiers a regarder :

`src/styles/tokens.css`

`src/components/Button.tsx`

`src/components/Button.module.css`

templates publics

pages publiques

- Ce qu'il faut verifier :

couleurs de bleu, boutons, typographie, variantes, etats hover et contrastes

- Methode de resolution pas a pas :
 1. definir une palette et une hierarchie typographique stables
 2. aligner les composants partages sur ces regles
 3. corriger les exceptions page par page
 4. documenter les usages autorises
- Definition de fini :

couleurs, polices et composants partages suivent un cadre unique

Reprendre la coherence visuelle et la qualite d'integration

- Impact : moyen
- Pourquoi : certains objets, images, centrages et espacements ne donnent pas la meme impression de finition d'un ecran a l'autre
- Fichiers a regarder :

pages publiques

templates

cartes

modales

- Ce qu'il faut verifier :

alignements, centrages, grille, comportement responsive, hauteur visuelle des blocs et positionnement des medias

- Methode de resolution pas a pas :
 1. auditer ecran par ecran les ecarts de centrage et d'espacement
 2. definir des regles communes de grille et d'alignement
 3. corriger les composants les plus reutilises en premier
 4. verifier desktop et mobile
- Definition de fini :

les principaux ecrans ont une integration visuelle homogene

Reprendre la gouvernance du front

- Impact : moyen
- Pourquoi : le projet a les briques d'un systeme front, mais pas encore des regles assez fermes sur quoi utiliser et comment
- Fichiers a regarder :

`src/styles/tokens.css`

`src/components/Footer.tsx`

`src/components/Button.tsx`

composants et pages fortement personnalisés

- Ce qu'il faut verifier :

liens placeholders, composants de navigation, patterns dupliques, exceptions non documentees

- Methode de resolution pas a pas :

1. definir les regles d'usage du socle front
2. lister les composants officiels
3. retirer les placeholders et cas non finis
4. documenter les choix pour la suite

- Definition de fini :

l'equipe sait quelle brique utiliser et les ecarts non voulus sont reduits

Passer les grosses routes en chargement differe

- Impact : moyen a eleve
- Pourquoi : le site charge trop de code des l'ouverture
- Fichiers a regarder :

`src/App.tsx`

- Methode de resolution pas a pas :
 1. identifier les routes lourdes
 2. passer configurateur, admin, templates et confirmations en chargement differe
 3. verifier les ecrans de fallback
- Definition de fini :

configurateur, admin, templates, commande et confirmation sont separees en chunks utiles

Supprimer les doublons de police et allger les medias

- Impact : moyen
- Pourquoi : la vitesse percue reste penalisee par des couts evitables
- Fichiers a regarder :

`index.html`

`src/styles/tokens.css`

pages avec hero et images

- Methode de resolution pas a pas :
 1. choisir une seule source de chargement police
 2. verifier les usages reels
 3. differer les medias non critiques

- Definition de fini :

une seule strategie police, medias non critiques differes

Poser le socle SEO par page

- Impact : moyen
- Pourquoi : le site a de la matiere, mais pas encore le cadre technique SEO
- Fichiers a regarder :

index.html

pages publiques

src/components/Footer.tsx

public/

- Methode de resolution pas a pas :
 1. definir title et description par page
 2. ajouter canonical et metas sociales
 3. generer robots.txt et sitemap.xml
 4. verifier les liens internes

- Definition de fini :

title, description, canonical, robots.txt , sitemap.xml et maillage propre

Corriger les points d'accessibilite les plus visibles

- Impact : moyen
- Pourquoi : labels, modales, focus et landmarks restent trop inegaux
- Fichiers a regarder :

src/pages/ContactPage.tsx

src/components/TemplateModal.tsx

src/components/OnboardingModal.tsx

src/components/PreviewModal.tsx

src/components/TemplateCardNew.tsx

- Methode de resolution pas a pas :
 1. remettre les labels visibles sur les formulaires
 2. corriger la gestion du focus dans les modales

3. verifier le clavier sur les cartes et boutons
4. refaire un passage Lighthouse accessibilite

- Definition de fini :

labels visibles, focus fiable, modales clavier correctes, contrastes repris

Harmoniser le systeme de design

- Impact : moyen
- Pourquoi : le socle existe deja, mais trop de styles en dur et de variables ambiguës restent en place
- Fichiers a regarder :

`src/styles/tokens.css`

pages publiques

admin

modales

- Methode de resolution pas a pas :
 1. retirer les doublons de variables
 2. reduire les styles inline
 3. clarifier les tokens de couleur, texte et ombre
 4. rapprocher l'admin et la vitrine
- Definition de fini :

tokens plus clairs, moins de styles inline, admin et vitrine mieux alignes

Chantier transverse bonnes pratiques

Ce que l'audit tech lead demande en plus

- une separation plus claire entre interface, logique metier et acces donnees
- une source de verite cote serveur pour les statuts sensibles
- des contrats de donnees plus stricts
- un niveau minimum de tests sur les flux qui vendent et encaissent
- une hygiene continue du lint au lieu d'un rattrapage occasionnel
- une configuration plus propre et moins de valeurs en dur

- moins de logs sensibles dans le navigateur
- un vrai cadre de gouvernance du front
- une meilleure coherence visuelle entre pages, templates et composants

Regle simple de travail

Pour chaque sujet code source :

1. confirmer le comportement actuel
2. corriger d'abord le risque business
3. tester tout de suite
4. seulement apres, nettoyer plus large